

Q1

VM - { hw2.1 }

Question

請以 Popek & Goldberg 提出的定理解釋何謂 關鍵(或稱臨界)指令(critical instruction)? 針對這類因硬體設計不週延所引發的問題, 利用二進制翻譯(binary translation)技術帶來的做法, 請敘述那兩種不用 source program 進行改善, 針對二進碼指令達到有效虛擬化的技術方案.

Answer

根據來源檔案, Popek 與 Goldberg 對指令集架構 (ISA) 虛擬化的定理, 以及針對關鍵指令 (critical instruction) 的技術解決方案說明如下:

1. 以 Popek & Goldberg 定理解釋「關鍵指令」

Popek 與 Goldberg (1974) 提出了判斷一個 ISA 是否能有效支援虛擬機器的充足條件。他們將與硬體互動的指令分為以下類別:

- **特權指令 (Privileged instructions)**: 當處理器處於使用者模式 (User mode) 執行時會產生 Trap (陷阱/中斷) 的指令。
- **敏感指令 (Sensitive instructions)**: 包含「控制敏感」和「行為敏感」兩類。
 - **控制敏感**: 企圖修改系統資源配置 (如記憶體映射或處理器模式) 的指令。
 - **行為敏感**: 其執行結果取決於資源配置 (如所在的實體位址或處理器模式) 的指令。

關鍵指令 (Critical instructions) 的定義: 根據 **Theorem 1**, 一個 ISA 只有在「敏感指令是特權指令的子集」時, 才是可以被有效虛擬化的。所謂的「**關鍵指令**」, 就是指那些「**屬於敏感指令, 但不是特權指令**」的指令^{**}。這類指令在使用者模式下執行時不會觸發 Trap^{**}, 因此虛擬機器監視器 (VMM) 無法直接攔截並模擬它們, 會導致虛擬機執行結果錯誤, 這是硬體設計不週延所引發的問題。

2. 針對二進碼指令的二進位翻譯 (Binary Translation) 技術方案

對於含有關鍵指令的非虛擬化架構（如 Intel IA-32），無需原始程式碼，可利用與動態二進位翻譯相關的技術來達成有效虛擬化。來源中提到的兩種主要技術方案為：

方案一：補丁技術 (Patching / Scanning and Patching)

這是處理「混合虛擬機器 (Hybrid VM)」最務實的做法，透過修改執行中的二進碼來實現。

- **做法**：VMM 在客體 (Guest) 程式執行前掃描其指令流。一旦發現「關鍵指令」，VMM 會將其替換 (Patch) 為一個 **Trap 指令 (系統呼叫) 或跳轉 (Jump) 指令**，使其強制將控制權轉移給 VMM。
- **優點**：原本不會產生 Trap 的關鍵指令現在會主動通知 VMM。VMM 接收到控制權後，會在軟體層級模擬該指令應有的行為，確保系統的等效性。

方案二：模擬碼快取 (Caching Emulation Code)

當關鍵指令出現頻率較高時，單純透過 Trap 處理開銷過大，可採用類似程序虛擬機的代碼快取機制。

- **做法**：VMM 不僅掃描單一指令，而是將關鍵指令及其周邊的指令區塊進行**二進位翻譯 (Binary Translation)**，轉換成一組專門針對該位置優化的模擬常式。
- **執行流程**：翻譯後的程式碼被存放在 **代碼快取 (Code Cache)** 中。後續程式執行到該處時，會直接透過跳轉表定位到快取中的翻譯塊，以原生速度執行模擬邏輯，避免頻繁觸發 Trap 所產生的 context switch 開銷。

總結來說，這兩者都直接作用於二進位執行檔 (Binary)，藉由「**掃描與替換**」或「**翻譯與快取**」，在軟體層級解決了硬體無法自動攔截敏感指令的問題。

Question

請說明 IBM 公司提出的 Interpretive Execution Facility (IEF) 是如何運作的？

Answer

IBM 公司提出的 **詮釋執行功能 (Interpretive Execution Facility, IEF)** 是一種將虛擬化功能從軟體層級 (VMM) 整合至處理器硬體的技術，其目的是為了大幅提升虛擬機器的執行效能，並使虛擬機器的行為更加可預測。

以下是 IEF 運作方式的詳細說明：

1. 核心指令：SIE (Start Interpretive Execution)

IEF 的運作主要圍繞著 **SIE 指令**。虛擬機監視器 (VMM) 軟體透過執行 SIE 指令，將處理器的控制權交給硬體的 IEF 部分，進入「詮釋執行模式」。

2. 運作流程

在執行 SIE 指令之前與期間，系統會進行以下操作：

- **準備狀態表**：VMM 軟體會先處理任何掛起的中斷，並設置一個指向**狀態描述表 (State Description Table)** 的指標。該表格包含了該虛擬機器的架構狀態（如暫存器數值等）。
- **載入資源**：處理器會根據狀態表載入必要的資源（例如通用暫存器）。
- **硬體直接執行**：一旦進入 IEF 模式，處理器會開始直接執行虛擬機器的指令流。**大多數的特權指令 (Privileged instructions) 會直接由硬體處理**，而不需要跳轉回軟體模擬。
- **例外處理**：I/O 指令以及少數複雜或不常見的指令通常不在硬體支援範圍內，仍需由軟體處理。

3. 暫停與退出機制 (Exit Mechanisms)

硬體詮釋執行會在特定情況下掛起，並將控制權交還給 VMM 軟體。主要分為兩類：

1. **中斷 (Interrupt)**：當產生一個針對 VMM 或其他虛擬機器的中斷時，IEF 會暫停。中斷處理完成後，控制權會回到原先的 SIE 指令處重新開始。
2. **攔截 (Interception)**：控制權會轉移到 SIE 之後的下一條指令。這發生於以下情況：
 - 遇到硬體不支援詮釋的指令（如 I/O 指令）。
 - 執行中發生異常 (Exception)。
 - 偵測到外部設置的攔截條件或特殊情況（例如客體作業系統進入等待狀態）。
3. 狀態更新

在控制權回到 VMM 軟體之前，IEF 硬體會自動更新包含虛擬機器狀態描述的表格，確保軟體能獲得最新的虛擬機狀態。

總結

IEF 透過硬體輔助功能（如計時器輔助、地址轉換繞過技術等），讓虛擬機器的執行速度極度接近原生平台的表現。根據來源數據，導入這類硬體輔助（VM Assists）後，虛擬機器的效能損耗可大幅降低。

Q3

VM - { hw2.3 }

Question

請簡述 INTEL 公司的 VT-x(Vanderpool)技術, 其優點何在?

Answer

Intel 公司的 **VT-x (Vanderpool)** 技術是針對 IA-32 處理器開發的硬體輔助虛擬化技術，其概念與 IBM 的詮釋執行（Interpretive Execution）功能相似，旨在提升虛擬機器的執行效能並簡化 VMM（虛擬機器監視器）的設計。

VT-x 技術簡述

VT-x 的核心是引入了一種新的處理器操作模式，稱為 **VMX 模式**。此模式下包含兩種子操作：

- **VMX 根操作 (VMX root operation)**：供 VMM 運作。其行為與一般處理器大致相同，但新增了一組 VMX 指令供 VMM 使用。
- **VMX 非根操作 (VMX non-root operation)**：供客體 (Guest) 虛擬機器運作。在此模式下，處理器對關鍵共享資源的控制權會受到限制，以確保 VMM 擁有最終管理權。

關鍵特性在於，這兩種操作模式都支援完整的 **IA-32 四個特權層級 (Ring 0-3)**。這使得客體作業系統核心可以運行在原本預期的 Ring 0，而應用程式運行在 Ring 3，消除了傳統虛擬化中「特權級壓縮 (Ring Compression)」的問題。

VT-x 的優點

根據來源，VT-x 技術帶來了以下主要優點：

1. **提升執行效率**：對於不涉及關鍵共享資源的程式碼區塊，硬體能以接近原生的速度執行。只有在少數必須由 VMM 介入的情況下才會發生「退出 (Exit)」，這大幅減少了從虛擬機回到監控器的次數。
2. **硬體輔助狀態管理**：VT-x 定義了一種稱為 **VMCS (虛擬機器控制結構)** 的資料結構，用以保存虛擬機器的架構狀態。硬體能自動完成狀態的載入與卸載，避免了 VMM 透過軟體執行大量 load/store 指令所產生的昂貴開銷。
3. **消除對準虛擬化 (Paravirtualization) 的依賴**：由於客體軟體可以在其原本設計的特權環中執行，VT-x 允許直接執行未經修改的標準版作業系統，無需像 Xen 等系統那樣需要修改 OS 核心程式碼。
4. **優化的異常處理**：透過 VMCS 中的「異常位圖 (Exception Bitmap)」，硬體可以直接將某些異常交付給客體系統處理，而無需由 VMM 攔截、檢查後再反射回客體 OS，減少了不必要的處理成本。
5. **靈活的行為設定**：VMM 可以透過控制位元（如「RDTSC 退出」或「使用 TSC 偏移」）來精確配置虛擬機器的行為（例如：讓虛擬機看到的系統時間僅為其實際運行的週期數），且不會產生額外的軟體效能懲罰。

Q4

VM - { hw2.4 }

Question

請簡述 動態二進制代碼重寫在安全性考量中透過 DynamoRIO 系統的經驗可以扮演那些角色？

Answer

根據來源檔案與 **DynamoRIO**（其演進自 RIO 系統與早期 HP Dynamo 系統）的經驗，動態二進制代碼重寫（Dynamic Binary Rewriting）在安全性考量中主要扮演「**程式護送（Program Shepherding）**」的角色。

透過在執行時即時檢查與轉換指令，該技術能為程式建立一個「沙盒（Sandbox）」，以防止潛在的攻擊並確保執行安全。以下是其扮演的關鍵角色與具體機制：

1. 限制控制權轉移 (Restricting Control Transfers)

DynamoRIO 系統能監控程式中所有的跳躍與分支指令，確保控制流始終處於合法狀態：

- **驗證跳躍目標：**系統會檢查每一個分支指令（包括條件分支與間接跳躍）的目標位址是否合法。
- **執行時檢查：**在詮釋執行階段或將代碼存入快取（Code Cache）之前進行檢查。如果是常見的「直接分支」，系統會在「鏈結（Chaining）」區塊時進行一次性安全檢查；如果是「間接跳躍」，則會在執行時進行動態檢查。
- **防止漏洞利用：**透過限制跳躍目標，可以有效阻斷如緩衝區溢位（Buffer Overflow）後竄改返回位址或函數指針的攻擊路徑。

2. 限制代碼執行 (Restricting Code Execution)

系統會對代碼的**來源（Origin）**進行驗證，確保只有經過許可的代碼才能執行：

- **代碼來源驗證**：在將指令複製到基本區塊快取（Basic Block Cache）之前，系統會驗證代碼是否來自磁碟上的合法區域，或者是否在載入後被惡意竄改。
- **動態生成代碼的管理**：對於合法生成的動態代碼（如 JIT 編譯），系統會進行認證；對於非法跳入可寫入記憶體空間（Writable memory）的行為，則會予以攔截。
- **監控自我修改代碼**：透過將原始代碼頁設為唯讀，系統能攔截所有試圖修改代碼的寫入行為，並在驗證其合法性後才允許重新翻譯與執行。

3. 系統自我保護與沙盒化 (Protecting the Runtime Monitor)

為了防止攻擊者繞過安全檢查，DynamoRIO 必須保護自身（Runtime）不被客體程式破壞：

- **分層權限管理**：系統使用不同的分頁保護設定。當處於「RIO 模式（系統管理）」時，權限較高（如代碼快取為可讀寫）；當切換到「應用程式模式（執行客體程式）」時，權限則降至最低。
- **記憶體隔離**：例如，在應用程式模式下，存放對映表（Map Tables）的記憶體會被設為**唯讀**，而代碼快取則設為**唯執行**，防止應用程式惡意修改安全原則或快取內容。

4. 效能與安全性的平衡

DynamoRIO 借鑒了動態二進制最佳化的經驗，將安全性檢查與**代碼快取（Code Cache）**技術結合：

- **攤銷檢查開銷**：許多安全檢查（如直接分支的目標驗證）只需在區塊鏈結時執行一次。
- **減少效能損耗**：透過執行經過優化且已驗證的超區塊（Superblocks），系統能在維持高度安全性的同時，減少因頻繁切換回模擬管理器進行檢查所產生的延遲。

Q5

Question

請簡述 Internet Suspend/Resume (ISR) 技術與 Stanford Collective 項目技術的主要功能,方法與目的. VMware 公司開發的 Vmotion 目前有那些限制?

Answer

根據來源檔案，**Internet Suspend/Resume (ISR)** 技術、**Stanford Collective** 項目以及 **VMware VMotion** 都是關於電腦運算環境遷移（Migration）的技術，以下分別簡述其功能、方法、目的及限制：

1. Internet Suspend/Resume (ISR)

- **主要功能與目的：**旨在讓使用者能在一台機器上「暫停」工作，移動到另一地點後，再透過網際網路從另一台機器「回復」執行。其目的是實現完全的環境行動力，讓使用者不需攜帶電腦也能隨時存取自定義的系統環境。
- **方法：**
 - **封裝（Encapsulation）：**將虛擬機器的狀態（包含記憶體與磁碟內容）封裝成一個檔案。
 - **分散式檔案系統：**將此狀態檔案存放在分散式檔案系統中，使其可透過網路在目的地存取。
 - **提取模型（Pull Model）：**目的地電腦在使用者登入時，根據需求從網路「提取」並加載環境狀態。
 - **Mobile IP：**利用 Mobile IP 技術來維持網路連線的透明性，避免因更換地點而需更動 IP 位址。

2. Stanford Collective 項目

- **主要功能與目的：**與 ISR 類似，目標是將包含作業系統與應用程式的完整「膠囊（Capsule）」遷移至目的地，並致力於提升傳輸效率，使環境遷移能在短時間內（如 20 分鐘的通勤時間）完成。
- **方法：**
 - **推送模型（Push Model）：**將膠囊直接發送至目的地機器。

- **記憶體氣球技術 (Ballooning)**：在客體 OS 中執行「氣球」程式來釋放不必要的記憶體分頁並將其歸零，以壓縮待傳輸的狀態大小。
- **增量磁碟 (Incremental Disks)**：利用寫入時複製 (Copy-on-write) 與階層架構，僅傳輸與現有膠囊之間的差異，而非完整磁碟映像。
- **哈希副本 (Hashed Copy)**：使用 SHA-1 哈希識別數據塊。若目的地已存在相同哈希值的區塊，則不需重複傳輸，大幅節省頻寬。

3. VMware VMotion 的限制

雖然 VMotion 是商業化的成熟產品，但在來源編寫時仍存在以下限制：

- **伺服器叢集限制**：來源與目的地主機必須位於同一個由 VirtualCenter 管理的伺服器叢集中。
- **共享儲存需求**：兩台主機必須擁有完全相同的檔案系統，且磁碟必須位於**儲存區域網路 (SAN)** 上的共享磁碟中。
- **硬體一致性**：兩台主機的處理器必須具有相同的架構，且必須由**同一供應商 (Vendor)** 提供，以避免狀態加載時的相容性問題。
- **應用程式類型**：目前僅限於執行單機 (Stand-alone) 應用程式，不支援執行多處理器叢集應用程式 (Multiprocessor cluster applications) 的虛擬機。
- **網路硬體**：需支援 Gigabit 乙太網路轉換器。

Q6

[VM - { hw2.6 }](#)

Question

簡介何謂 recursive virtualization(遞歸虛擬化)? Popek and Goldberg 對傳統第三代計算機要實現 recursive virtualization 需要那些條件?

Answer

遞歸虛擬化 (Recursive Virtualization) 是指在一台虛擬機器中再次執行虛擬機器系統的技術。簡單來說，這意味著虛擬機器監視器 (VMM) 本身作為一個虛擬機器，在另一

個處於特權模式 (Privileged mode) 的 VMM 控制下，運行於使用者模式 (User mode)。

根據 **Popek and Goldberg** 的研究，針對傳統第三代計算機實現遞歸虛擬化，需滿足以下兩大核心條件（正式定義於其第二定理）：

1. **系統必須是可虛擬化的 (Virtualizable)**：該計算機架構必須首先滿足其第一定理，即所有的敏感指令 (Sensitive instructions) 都必須是特權指令 (Privileged instructions)，確保 VMM 能夠攔截所有關鍵操作。
2. **必須能建構出無時間依賴性 (No timing dependences) 的 VMM**：
 - 由於 VMM 運行在另一個虛擬機器中，效能可能會受到影響。
 - 如果 VMM 的行為依賴於精確的時間關係（例如計時器），當其在使用者模式下執行時，效能的波動可能導致其行為與原生平台不一致，從而違反虛擬化的「等效性 (Equivalence)」原則。

Popek and Goldberg 的第二定理 (Theorem 2) 總結如下：一個傳統第三代計算機是可遞歸虛擬化的，若且唯若：

- (a) 它是可虛擬化的。
- (b) 可以為其建構一個沒有任何時間依賴性的 VMM。

在實際應用中，遞歸虛擬化通常很少超過兩層，因為每一層 VMM 都會消耗系統資源（特別是記憶體），隨著層數增加，可用資源會迅速萎縮。

Q7

[VM - { hw2.7 }](#)

Question

簡述在兩servers間gues程式如何進行live migration?

Answer

在兩台伺服器之間進行客體（Guest）程式的**即時遷移（Live Migration）**，主要是透過將虛擬機器的完整執行環境封裝成一個「**膠囊（Capsule）**」並在網路間傳輸來達成。

以下是根據來源檔案（以 **VMware VMotion** 為代表）簡述的執行流程與關鍵技術：

1. 即時遷移的執行流程

遷移過程旨在將執行中的虛擬機器狀態從來源主機搬移到目標主機，且對使用者而言幾乎是無感的：

- **基準拷貝 (Baseline Copy)**：在客體程式持續執行的情況下，系統先將其大部分的記憶體資料拷貝到目標伺服器。
- **暫時掛起 (Suspension)**：當大部分資料拷貝完成後，來源主機短暫暫停虛擬機器的執行。
- **傳輸最後狀態**：將剩餘變動的記憶體分頁，以及處理器狀態（暫存器、PC等）、BIOS、裝置狀態（如網卡 MAC 位址）等核心資訊傳送到目標主機。
- **恢復執行 (Resumption)**：在目標伺服器上啟動虛擬機器，並從剛才暫停的點繼續執行。

2. 提升效率的最佳化技術

為了縮短傳輸時間與減少頻寬占用（例如在 Stanford Collective 項目中），會使用以下技術：

- **記憶體氣球技術 (Ballooning)**：在遷移前，透過客體 OS 內的氣球程式釋放不必要的記憶體頁面並歸零，以壓縮待傳輸的狀態大小。
- **增量磁碟 (Incremental Disks)**：利用「寫入時複製（Copy-on-write）」技術，僅傳輸與現有磁碟映像之間的差異部分。
- **哈希副本 (Hashed Copy)**：利用 SHA-1 等算法識別數據塊。如果目標伺服器上已存在相同哈希值的區塊（例如相同的作業系統檔案），則不需重複傳輸該區塊。

3. 維持網路連線與相容性

- **透明性維持**：為了讓程式在遷移後仍能維持原本的網路連線，通常會結合 **Mobile IP** 技術或透過虛擬私人網路（VPN）建立**隧道（Tunneling）**，確保 IP 位址在搬移後依然有效。
- **硬體限制**：以商業化產品 VMotion 為例，通常要求來源與目標伺服器必須位於同一叢集、使用相同的共享儲存裝置（如 SAN），且處理器架構必須一致，以避免指令相容性問題。

Q8

VM - {hw2.8}

Question

簡述何謂 Paravirtualization? 在此情況下, XEN 提供了那些改進?

Answer

準虛擬化 (Paravirtualization) 是一種虛擬化技術，其核心概念是透過修改客體作業系統（Guest OS）的原始碼，使其意識到自己正執行於虛擬化環境中，進而與虛擬機器監視器（VMM）進行協作，以繞過底層指令集架構（ISA）中難以虛擬化的限制。

在此架構下，**Xen 系統** 針對 IA-32 架構（x86）提供了以下關鍵改進與技術特性：

1. 解決非虛擬化指令問題

- IA-32 指令集中包含一些「關鍵指令（Critical instructions）」，這些指令在使用者模式執行時不會觸發 Trap，導致傳統虛擬化難以攔截。
- **Xen 的做法**：它定義了一個與底層硬體相似但不完全相同的虛擬機介面。客體作業系統中原本執行特權操作的部分會被替換為對 VMM 的顯式呼叫（Hypercalls），由 VMM 驗證並代為執行。

2. 特權層級的重新配置 (Ring Re-mapping)

- 傳統 IA-32 系統中，OS 核心執行於 Ring 0，應用程式執行於 Ring 3。

- **Xen 的改進**：它利用了 IA-32 具備四個特權環（Ring 0-3）的特性，將 **VMM 運行在 Ring 0**，並修改客體 OS 核心使其執行於 **Ring 1**，而應用程式仍維持在 Ring 3。這種做法避免了 OS 與 VMM 之間的特權衝突。

3. 簡化虛擬化任務與提升效能

- **消除複雜轉換**：由於 OS 已被修改以配合 VMM，Xen 不再需要進行複雜的「掃描與補丁（Scanning and Patching）」或是維護龐大的「陰影分頁表（Shadow Page Tables）」，大幅降低了軟體模擬的開銷。
- **效能優勢**：根據研究，經過 Xen 準虛擬化修改後的 Linux，其變動程式碼量僅不到 3000 行（約 1.36%），但在各項基準測試中的效能可達到原生系統（Native）的 **90% 以上**。

4. 維持應用程式相容性

- 儘管 Xen 需要修改作業系統核心（如 Linux 或 Windows XP），但其設計目標是讓 **應用程式二進位檔（Application Binaries）無需任何修改**即可直接在虛擬環境中執行。

總結來說，Xen 透過「修改 OS 核心以換取效能」的策略，成功地在原本難以虛擬化的 IA-32 硬體上，實現了極低損耗的虛擬化環境。